BTS CIEL · PROGRAMMATION PHP · TRAVAUX PRATIQUES

TP2 — Register & Login : activer l'espace membre de notre boutique

Cours associé : `01_Cours/Cours_PHP.pdf` *(sections 8, 9, 11, 12 et 13).*

Contexte

Front-end déjà réalisé — HTML · CSS · JavaScript

- Mode clair/sombre
- Mise en page responsive
- Accueil et boutique
- Formulaires d'inscription et de connexion
- Panier et récapitulatif de commande

Back-end à développer — PHP · Sessions · MariaDB · SQL

- Traitement des formulaires
- Gestion de l'authentification
- Récupération des produits
- Gestion du panier et de ses quantités
- Enregistrement et consultation des commandes

Objectif : rendre fonctionnels les formulaires HTML déjà réalisés dans le site Boutique. Le TP progresse en trois étapes :

Étape	Parties	Résultat
1	1 à 4	Les formulaires fonctionnent, mais les comptes ne sont pas conservés
2	5	L'utilisateur reste connecté (sessions)
3	6 à 8	Comptes, produits, panier et commandes enregistrés dans MariaDB

Étape 1 — Traiter les formulaires (identifiants de test)

Partie 1 — Register : traiter les données reçues

Utiliser la page `register.php`.

Travail demandé :

1. Détecter l'envoi du formulaire avec `$_SERVER["REQUEST_METHOD"]`.
2. Récupérer l'identifiant et le mot de passe avec `$_POST`.
3. Vérifier côté serveur que les champs ne sont pas vides.
4. Vérifier que le mot de passe respecte les règles demandées : au moins 8 caractères, une majuscule, un chiffre (`mb_strlen()` ou expression régulière avec `preg_match()`, section 8).
5. Afficher un message de réussite ou **toutes** les erreurs rencontrées.

Notions travaillées : `$_POST`, `??`, `trim()`, `if`, `preg_match()`.

Remarque : les attributs HTML `required` et les regex JavaScript ne remplacent pas la validation côté serveur. Les données envoyées au serveur doivent être vérifiées à nouveau en PHP.

Partie 2 — Login : vérifier les identifiants

Le formulaire HTML de connexion est déjà disponible dans la page `login.php`.

> Si le front-end de la Boutique n'est pas disponible, utiliser la page de connexion complète fournie en **annexe A**.

- récupérer l'identifiant et le mot de passe ;
- comparer les valeurs à des identifiants de test définis dans le code PHP ;
- afficher « Connexion réussie » si les deux correspondent ;
- afficher un message **générique** si les identifiants sont incorrects.

Exemple :

```php
<?php
$message = "";

if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $identifiant = trim($_POST["identifiant"] ?? "");
    $motDePasse  = $_POST["mot_de_passe"] ?? "";

    // Identifiants de test
    $identifiantCorrect = "etudiant";
    $motDePasseCorrect  = "php12345";

    if (
        $identifiant === $identifiantCorrect &&
        $motDePasse === $motDePasseCorrect
    ) {
        $message = "Connexion réussie !";
    } else {
        $message = "Identifiant ou mot de passe incorrect.";
    }
}
?>

<p><?= htmlspecialchars($message, ENT_QUOTES, "UTF-8") ?></p>
```

Identifiants de test : `etudiant` / `php12345`.

- Compléter les conditions et tester plusieurs cas : bons identifiants, mauvais mot de passe, mauvais identifiant, champs vides, espaces autour de l'identifiant.
- Ajouter un lien de retour vers l'accueil et sécuriser tous les affichages avec `htmlspecialchars()`.

> **Remarque :** ce code ne constitue pas un système d'authentification utilisable en production : le mot de passe est écrit en clair dans le code et aucune identité n'est conservée après la réponse du serveur.

Partie 3 — Faire le lien entre register et login

On constate que les identifiants saisis dans `register.php` ne sont pas automatiquement disponibles dans `login.php`.

1. Pourquoi les données reçues par `register.php` ne sont-elles pas automatiquement conservées ?
2. Comment pourrait-on transmettre les données d'une page à une autre ?
3. Quelle différence y aurait-il entre stocker les données dans une variable PHP et les conserver durablement ?

Partie 4 — Sécurité et recul

- Pourquoi faut-il valider les données en PHP même si le formulaire contient `required` ?
- Pourquoi ne faut-il pas afficher directement une donnée utilisateur dans une page HTML ?
- Pourquoi le mot de passe ne devrait-il pas être stocké en clair dans une véritable application ?
- Pourquoi le système de connexion actuel ne permet-il pas de rester connecté ?

Étape 2 — Rester connecté avec les sessions

Partie 5 — Sessions

1. À la connexion réussie : `session_start()`, `session_regenerate_id(true)`, puis stocker l'identifiant dans `$_SESSION["utilisateur"]` et rediriger vers `compte.php` (`header()` + `exit`).
2. Créer `compte.php`, **réservée aux utilisateurs connectés** : redirection vers `login.php` sinon.
3. Créer `logout.php` qui détruit la session.
4. Dans l'en-tête du site (`header.php`), afficher « Connecté en tant que … » + lien **Se déconnecter**, ou les liens **Connexion / Inscription** si personne n'est connecté.
5. Pour faire le lien entre `register` et `login` sans base de données : à l'inscription, stocker le compte dans `$_SESSION["comptes"]` avec un mot de passe **haché** (`password_hash()`), et vérifier la connexion avec `password_verify()`. Quelle est la limite de ce stockage ?

Étape 3 — Enregistrer les données dans MariaDB

Partie 6 — Comptes utilisateurs en base de données

1. Créer la base `boutique` et la table `utilisateurs` (section 12), puis le fichier `db.php` (connexion PDO).
2. **Register** : après validation, vérifier que l'identifiant n'existe pas, puis insérer `identifiant` + `password_hash(...)` avec une requête préparée. Gérer l'erreur « identifiant déjà utilisé ».
3. **Login** : récupérer l'utilisateur avec une requête préparée, vérifier avec `password_verify()`, puis ouvrir la session (partie 5).
4. Dans phpMyAdmin, consulter la table : que contient la colonne du mot de passe ? Inscrire deux comptes avec le **même** mot de passe : les valeurs stockées sont-elles identiques ? Pourquoi ?
5. Tenter une injection SQL (`' OR '1'='1`) dans le champ identifiant. Pourquoi échoue-t-elle ?

Partie 7 — Produits et panier

1. Créer la table `produits` (section 12) et y insérer au moins 5 produits.
2. Afficher les produits de la boutique depuis la base (`SELECT`), prix formatés avec `number_format()`, valeurs échappées.
3. Gérer le **panier** dans la session sous la forme `$_SESSION["panier"][id_produit] = quantité` :

```php
$id = filter_var($_POST["produit_id"] ?? null, FILTER_VALIDATE_INT);

if ($id !== false && $id !== null) {
    $_SESSION["panier"][$id] = ($_SESSION["panier"][$id] ?? 0) + 1;
}
```

> Un identifiant entier n'est pas forcément un produit existant : avant l'ajout, vérifier que le produit **existe en base** (requête préparée `SELECT id FROM produits WHERE id = :id`).

4. Page `panier.php` : lister les produits du panier (le **prix est relu en base**, jamais pris dans le formulaire), permettre de modifier les quantités (entier ≥ 1) et de supprimer une ligne, calculer le **total**.
5. Pourquoi ne faut-il jamais faire confiance au prix envoyé par le navigateur ?

Partie 8 — Commandes (extension)

1. Créer les tables `commandes` (`id`, `utilisateur_id`, `date_commande`, `total`) et `lignes_commande` (`commande_id`, `produit_id`, `quantite`, `prix_unitaire`).
2. À la validation du panier (utilisateur **connecté** obligatoire) : enregistrer la commande et ses lignes dans une **transaction** (`beginTransaction()`, `commit()`, `rollBack()` en cas d'erreur), puis vider le panier.
3. Créer `mes_commandes.php` : l'historique des commandes de l'utilisateur connecté (et **seulement les siennes**).
4. Pourquoi enregistre-t-on `prix_unitaire` dans `lignes_commande` plutôt que de relire le prix du produit ?

Critères de réussite

- ☐ Toutes les données reçues sont **validées en PHP**.
- ☐ Tous les affichages de données utilisateur sont **échappés** (`htmlspecialchars()`).
- ☐ Aucune requête SQL ne concatène une saisie : requêtes **préparées**.
- ☐ Les mots de passe sont **hachés** (`password_hash()`), jamais stockés en clair.
- ☐ Les pages privées vérifient la session ; la session est détruite à la déconnexion.
- ☐ Les messages d'erreur de connexion sont **génériques**.
- ☐ Les erreurs PHP ne sont pas affichées à l'utilisateur dans la version finale.

Annexe A — Page de connexion complète (identifiants de test)

Page autonome à utiliser pour la partie 2 si le front-end de la Boutique n'est pas disponible.

```php
<?php
$message = "";

if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $identifiant = trim($_POST["identifiant"] ?? "");
    $motDePasse = $_POST["mot_de_passe"] ?? "";

    // Identifiants de test
    $identifiantCorrect = "etudiant";
    $motDePasseCorrect = "php12345";

    if (
        $identifiant === $identifiantCorrect &&
        $motDePasse === $motDePasseCorrect
    ) {
        $message = "Connexion réussie !";
    } else {
        $message = "Identifiant ou mot de passe incorrect.";
    }
}
?>

<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <title>Connexion</title>
</head>
<body>
    <h1>Se connecter</h1>

    <form method="post">
        <label for="identifiant">Identifiant :</label>
        <input type="text" id="identifiant"
               name="identifiant" required>

        <br><br>

        <label for="mot_de_passe">Mot de passe :</label>
        <input type="password" id="mot_de_passe"
               name="mot_de_passe" required>

        <br><br>

        <button type="submit">Se connecter</button>
    </form>

    <p>
        <?= htmlspecialchars($message, ENT_QUOTES, "UTF-8") ?>
    </p>
</body>
</html>
```

Identifiants de test : `etudiant` / `php12345`.